

Point-and-Click Reproducibility for Command-Line Data Analysis

Anthony Westbrook
University of New Hampshire
Durham, USA
Anthony.Westbrook@unh.edu

Bocheng Cui
University of New Hampshire
Durham, USA
Bocheng.Cui@unh.edu

Lakshmi Ongolu
University of New Hampshire
Durham, USA
Lakshmi.Ongolu@unh.edu

Elizabeth Varki
University of New Hampshire
Durham, USA
varki@unh.edu

Abstract

In data-intensive research, a file rarely exists in isolation: a single source file typically implies a lineage of derived files, from processing raw satellite imagery to analyzing brain scans. While point-and-click consumer applications maintain these relationships automatically (e.g., photo software generating thumbnails), exploratory scientific computing is primarily command-line driven, and the dependencies between source and derived files are managed by the researcher. Build systems and workflow managers can automate this, but only once dependencies are explicitly declared, a step such analysis rarely takes, leaving the manual loop prone to error and state drift, where derived files become stale relative to their source.

To address this, we introduce point-and-click reproducibility for exploratory command-line data analysis, and implement it in RepeatFS, a file system where derived files are Virtual Dynamic Files (VDFs). By constraining derivation to a predefined set of valid operations and recording it in the file path, the model makes any accessible file the consistent, reproducible output of a valid workflow. A VDF is a transient store kept consistent with its source and the computations that produced it; its lineage is scaffolded as a virtual directory tree, so standard shell commands become menu actions: listing a directory opens a menu, reading a file clicks an item, and traversal chains them. The structure of the path is the structure of the workflow, executed just-in-time whenever the accessed file is missing or stale.

By bringing this automation to the command line, VDFs make the reproducibility of single-source derived files an intrinsic, transparent property of the file system rather than an afterthought. Our evaluation shows that RepeatFS's in-memory streaming architecture avoids repeated disk materialization, achieving up to a 4.5× speedup over traditional file-based workflows.

CCS Concepts

• **Software and its engineering** → **File systems management**; • **Information systems** → **Data provenance**; *Storage management*.

Keywords

Point-and-Click Reproducibility, File Systems, Virtual Dynamic Files, Provenance, Data Lineage, Reproducibility, Exploratory Data Analysis

ACM Reference Format:

Anthony Westbrook, Lakshmi Ongolu, Bocheng Cui, and Elizabeth Varki. 2026. Point-and-Click Reproducibility for Command-Line Data Analysis. In *ACM Conference on Reproducibility and Replicability (ACM REP '26)*, July 20–22, 2026, Delft, Netherlands. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3820002.3828586>

1 Introduction

Raw data is rarely scientifically useful in its initial state. Instead, a single *source* file typically serves as the starting point for a lineage of *derived* files, each required to support analysis, visualization, or archival. Unless stated otherwise, these artifacts are realized as files in the file system.

These dependencies often follow structured patterns. Common structures include the *linear chain*, where a file undergoes a sequence of transformations (e.g., an Earth science workflow calibrating `scene.raw` → `scene.rad` → `scene.ndvi`), and the *fan-out dependency*, where one source drives multiple concurrent outputs (e.g., a server log producing error reports, audit trails, and traffic statistics). In practice, workflows in domains such as neuroimaging and genomics combine these patterns into deep dependency structures, where derived files depend on both their source data and the computations that produced them [1, 9, 12].

However, there is a fundamental disconnect between the structured dependencies among files and the process followed by researchers. While a final file may reflect a clean, linear lineage, the workflow used to discover it is rarely linear. Determining the correct sequence of operations to generate a derived file involves trial, error, and backtracking [14, 33]. Consider a researcher analyzing a raw server log (`access.csv`). To prepare the data for analysis, the researcher applies a set of atomic operations such as filtering for errors, anonymizing user identifiers, and sorting by timestamp. While the valid transformation rules are known, the effective *sequence* must be discovered empirically. For example, the researcher may initially filter for errors, only to realize that anonymizing identifiers first obscures necessary information. They must then backtrack, reverse the operations, and explore an alternative path.

In this setting, the workflow is not a fixed pipeline but a navigable state space. Multiple intermediate files may be generated:



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ACM REP '26, Delft, Netherlands

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2778-8/26/07

<https://doi.org/10.1145/3820002.3828586>

```
access.csv/sort/head
access.csv/head/sort
```

yet only a subset represent valid scientific results. This discrepancy between the clean structure required for reproducible lineage and the exploratory nature of command-line analysis introduces several failures of reproducibility [3, 20]:

- (1) *Input Error*: In ad hoc shell sessions, users may inadvertently apply incorrect transformations by mistyping parameters, specifying incorrect flags, or making similar mistakes. These errors can generate files that appear valid, yet differ from the results the researcher intended to produce.
- (2) *Provenance Loss*: Even when results are correct, the sequence of operations used to derive a file is often lost or fragmented across shell history, making it difficult or impossible to reconstruct [5, 17].
- (3) *Workspace Pollution*: Exploratory analysis generates a sprawl of intermediate files. Identifying dead-end pathways and deciding which files to retain imposes cognitive overhead during analysis and long-term maintenance.
- (4) *State Drift*: When a source file changes, dependent files go stale unless regenerated. Absent a maintained dependency graph, this falls to the user, and stale results persist unnoticed.

These failures are not independent; each is a symptom of a single underlying gap, namely, derivation lineage that is neither captured nor enforced during exploratory work, a single shared cause that one mechanism can remove.

Existing approaches such as shell scripts, workflow engines, and computational notebooks assume that workflows are specified *a priori*, rather than discovered interactively [15, 16]. As a result, exploratory command-line analysis is poorly supported by current reproducibility tools, which are oriented toward already-specified pipelines.

By contrast, consumer software routinely automates relationships between source data and derived results. In photo management applications, for example, thumbnails and filtered images are generated just in time in response to user interaction, with intermediate data hidden from view. Users can explore alternatives freely while persisting only the results they value.

The challenge is to bring this consumer-grade automation to command-line environments during the exploratory research phase. Our goal is to develop an application-independent system that reproduces the click-to-derive behavior of consumer software using only standard file system interactions. To the user, listing a directory (`ls`) should be equivalent to opening a menu, reading a file (`cat`) to clicking an action, and changing directories (`cd`) to chaining those actions into a workflow.

In this paper, we introduce point-and-click reproducibility, a model that constrains exploratory command-line workflows to valid, registered operations and records each in the file path, so that any file a user reaches is, by construction, the consistent output of a reproducible workflow. We present **RepeatFS**, a file system that captures lineage as it is discovered while supporting backtracking and branching.

2 The Point-and-Click Model

To address the reproducibility failures of ad-hoc command-line exploration, we propose a new interaction paradigm called point-and-click reproducibility. This model draws inspiration from the structural constraints imposed by Graphical User Interfaces (GUIs) [25]. In a GUI application (e.g., Photoshop or Excel), users cannot manually type incorrect syntax or invoke nonexistent functions; instead, they are restricted to selecting from a predefined set of valid operations. As a result, derived files are constructed only through sequences of valid interactions. Our goal is to enforce comparable constraints within the command-line environment, without sacrificing flexibility.

Realizing this goal presents a fundamental paradox: the model is named for a visual interaction pattern, yet its target users are computational researchers and data scientists who operate primarily within a text-based Command Line Interface (CLI). This constraint gives rise to two interdependent design challenges:

- (1) The User Interface Challenge: How can the browsing and selection metaphors of a GUI be projected onto the text-based CLI of a POSIX shell?
- (2) The System Challenge: How can an execution engine realize derived files just-in-time while preserving the semantics that standard file operations expect?

2.1 The User Interface

To bridge the gap between the visual point-and-click model and the textual command line, we rely on a structural isomorphism between the hierarchical action menus of a GUI and the directory trees of a file system. In this design, we map the three fundamental user actions of the point-and-click model - *Discovery*, *Selection*, and *Chaining* - directly to standard POSIX file operations [11].

2.1.1 Discovery via Listing (`ls` $\approx$ *Opening a Menu*). In a GUI, a user reveals available actions, for example by opening a context menu. In our design, the file system reveals valid transformations as *virtual directory entries*. We introduce a syntactic delimiter (e.g., the `+` symbol) that allows any file, source or derived, to act as a directory root. Consider the source file `data.csv` from Figure 1. When a user lists its virtual root (e.g., `ls data.csv+`), the system dynamically projects the corresponding derived files:

```
data.sort.csv, data.ksort.csv, data.grep.csv, data.xlsx
```

These entries represent the “menu” of predefined valid atomic operations for the `.csv` file type (we revisit how this set stays manageable in Section 7.3).

2.1.2 Selection via Access (`cat` $\approx$ *Clicking*). In a GUI, clicking a menu item triggers an immediate computation. In the CLI, the explicit signal of user intent is the file access system call that reads a file. When a user executes a command that reads a virtual file (e.g., `cat data.csv+/data.sort.csv`), the system interprets this access as a “click” and immediately computes the corresponding derived file, `data.sort.csv`, on demand.

2.1.3 Chaining via Traversal (`cd` $\approx$ *Drilling Down*). Complex workflows require chaining multiple operations. In our design, chaining is achieved through recursive directory traversal. Every derived

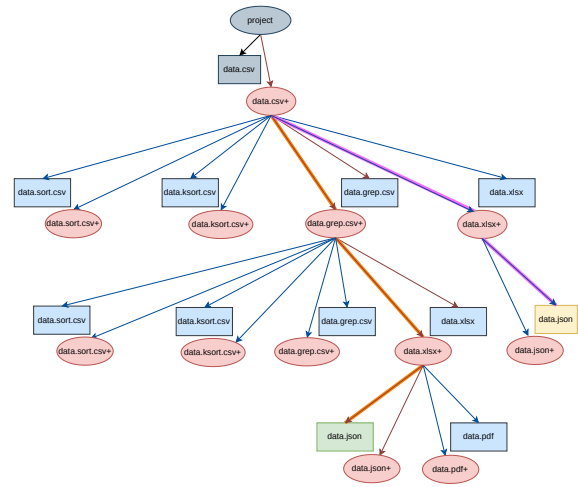


Figure 1: Point-And-Click mapped to directory traversal

virtual file projects its own virtual directory containing valid transformations for its specific file type. For example, a user can drill down into a lineage by navigating the path:

```
cd data.csv+/data.grep.csv+/data.xlsx+/
```

Here, the user has traversed into a derived Excel file's directory. Listing this virtual directory (`ls`) reveals predefined transformations valid for the `.xlsx` format:

```
data.json, data.pdf
```

Figure 1 depicts this virtual directory tree. By navigating from CSV to Excel to JSON, the user effectively constructs a multi-step pipeline simply by walking the file system. The structure of the path is the structure of the workflow.

2.2 Formal Model and Terminology

The interface described above implies a data model. We now formally define the components of point-and-click reproducibility.

2.2.1 File Artifacts. The model distinguishes between two classes of files:

- Source File: A disk file that serves as the root of a lineage (e.g., `data.csv`).
- Derived File: A file produced by applying a transformation to a source or another derived file: `data.csv` → `data.grep.csv`, `data.grep.csv` → `data.xlsx`.

2.2.2 Atomic File Transformations. Atomic transformations are pre-defined rules for converting one file type to another using a fixed command template. The system supports two execution patterns, distinguished by how a transformation directs its output: stream-based, written to standard output, and file-based, written to an explicit output file (e.g., `ssconvert`). Table 1 lists some transformations.

A single tool may be registered as several distinct transformations, each binding a fixed set of arguments to its own output suffix. For example, `sort` is available both as a plain lexical sort

Input Type	Output Suffix	Tool	Command Template
.csv, .txt	.sort.csv, .sort.txt	sort	sort {input} > {output}
.csv, .txt	.ksort.csv, .ksort.txt	sort	sort -k1,1 {input} > {output}
.csv	.xlsx	ssconvert	ssconvert {input} {output}
.csv, .txt	.grep.csv, .grep.txt	grep	grep 'Err' {input} > {output}
.xlsx	.json	xlsx2json	xlsx2json {input} > {output}

Table 1: Atomic transformations defined for various file types. Note the output-naming distinction: generic text tools preserve the input extension (e.g., `.csv` → `.sort.csv`), while converters like `ssconvert` change it (e.g., `.csv` → `.xlsx`).

(`.sort.csv`) and as a key-ordered sort (`.ksort.csv`, invoking `sort -k1,1`). Each parameterization is a separately named, separately discoverable entry, and the chosen variant is recorded in the path by its suffix, so a parameter choice is reproducible by construction: the suffix that names the transformation also names its arguments. Therefore, no separate parameter-tracking machinery is required.

The model admits only a predefined set of atomic transformations, just as a GUI restricts users to its defined actions. Therefore, arbitrary parameters typed at the command line are deliberately not admitted; permitting them would reintroduce the unconstrained typing the model replaces. A new parameterization is made available by registering it in the configuration as an additional transformation; one-off operations outside this curated set are performed directly in the shell.

2.2.3 Execution Lineage. An execution lineage is an ordered sequence of atomic transformations applied to a source file. In the point-and-click model, this lineage is explicitly encoded in the directory path. *Example:* Consider the path:

```
data.csv+/data.grep.csv+/data.xlsx+/data.json
```

This path represents a lineage where three atomic transformations are executed in sequence.

- (1) Apply `grep` to the physical source `data.csv`.
- (2) Apply `ssconvert` to the output of Step 1 to generate an Excel file (`.xlsx`).
- (3) Apply `xlsx2json` to the `.xlsx` file to produce the final `.json` output.

2.2.4 Ephemerality and Persistence. To support unrestrained exploration without workspace pollution, the model enforces a strict separation between viewing data and persisting data. All files within a virtual directory are ephemeral projections, meaning a user can explore infinite branching lineages without consuming physical disk space.

For example, in Figure 1, the user has explored two distinct paths, resulting in two separate lineages for `data.json`. Retention of these results is opt-in. To persist a specific result, the user must explicitly materialize the virtual file to physical storage using standard file operations such as a `copy` command:

```
cp data.csv+/data.grep.csv+/data.xlsx+/data.json data.json
```

The resulting file, `data.json`, is an ordinary disk file, but its derivation remains recoverable from RepeatFS's operation-level provenance record (Section 7.5). Until it is copied, the `data.json` located at the second virtual path

```
~/project/data.csv+/data.xlsx+/data.json
```

remains a transient, dynamically managed artifact.

For readability, the examples in this paper use compact file names that indicate only the latest transformation (e.g., `data.json`), with the full lineage carried by the path; the same name can therefore denote distinct files under different paths. Although our current implementation verbosely embeds the entire history directly into the file name, we are transitioning to this streamlined, path-dependent scheme.

Definition: Point-and-Click Reproducibility.

A reproducibility model where atomic transformations are fixed, but the execution lineage is constructed dynamically via user navigation. In this model, the file system serves as a state-space explorer: listing a directory reveals potential transformations, and accessing a file materializes the result transiently just-in-time. Because the lineage is encoded in the path, any accessible file is, by construction, the consistent output of a reproducible workflow.

The Necessity of a New Abstraction.

Realizing this model in a general-purpose computing environment requires a fundamental shift in system design. A derived file is dependent on its source. But standard file systems (e.g., `ext4`, `NTFS`) are designed to manage static, independent files. They lack the semantic awareness to enforce atomic transformations or the dynamic capability to materialize content just-in-time. To resolve the System Challenge and operationalize the point-and-click model, we cannot simply build a tool on top of the file system; we must change the behavior of the file itself.

3 The Virtual Dynamic File (VDF)

We propose a shift in data management: rather than managing derived files as static disk files, every derived result should be a **Virtual Dynamic (Derived) File (VDF)**, a novel file system abstraction for Point-and-Click Reproducibility. The notation reflects its dual nature: “Dynamic” refers to its ephemeral, just-in-time realization, while “Derived” defines its functional role as a result computed from a parent source.

To the user, a VDF appears as a standard file accessible via standard POSIX system calls (solving the User Interface Challenge). However, unlike a standard file, which is a passive store, a VDF is an active object that guarantees the following properties:

- (1) Just-in-Time Materialization (Transience): A VDF does not exist on disk. Its content is generated only upon access (via the `read()` system call). This enforces the “no workspace pollution” guarantee.
- (2) Automated Consistency: A VDF is tightly coupled to its source. If the source file changes, the next access to its VDF is guaranteed to trigger a re-computation, ensuring the result is always consistent with the current state of the data.
- (3) Stream-Based Concurrency: To ensure performance, a VDF is materialized via a pipeline. As the first transformation in the lineage produces output, it is immediately piped to the input of the next. The VDF exposes the final byte stream to the user as it is being computed, minimizing latency.
- (4) Dual I/O Semantics: While many tools accept streamed input, some require seekable file handles (e.g., `ssconvert`). A VDF intelligently adapts its behavior, functioning as a standard

pipe for stream-aware tools or simulating a physical block device for tools that require random access.

- (5) Immutability: To preserve the integrity of the lineage, a VDF is read-only. Users cannot manually edit a derived file; they can only alter the result by modifying the source or selecting a different transformation path.

4 Implementation

We implemented the point-and-click model in RepeatFS¹, our FUSE-based [26] provenance system [28, 29]. Figure 2 depicts its internal routing architecture.

By leveraging FUSE, RepeatFS mounts as a POSIX-compliant file system, allowing users, shell scripts, and executables to interact with both physical files and dynamic VDFs without requiring any modifications to the operating system kernel or the applications themselves. The architectural component facilitating this integration is the *Routing Layer*. Operating beneath the FUSE callback interface, the Routing Layer intercepts all file system calls issued by the kernel and intelligently routes them to either the underlying physical file system or the VDF Engine (Section 4.1).

To route incoming requests, the Routing Layer must distinguish between physical files and VDFs. It employs two distinct routing mechanisms depending on the type of system call:

- Path-Based Resolution (`getattr`, `open`): When the kernel provides a target path, the Routing Layer delegates resolution to an internal `FileEntry` object. To ensure physical files containing the `+` symbol remain accessible, the system first checks for match on disk. If found, the physical file is served normally. Only if this check fails does the system fall back to virtual routing: if the path contains a `+` delimiter, its components are cross-referenced against the Transformation Dictionary (Section 4.1.1). If the suffix matches a registered template, the target is classified as a virtual file. If both physical and virtual resolutions fail, the system rejects the request with a standard “No such file or directory” (`ENOENT`) error.
- Descriptor-Based Lookups (`read`, `write`): During the initial open call, the system determines if the target is a physical disk file or a virtual VDF. It saves this classification and assigns it a unique integer ID (the file descriptor). All later read and write requests only provide this integer ID. The Routing Layer uses this ID to look up the file’s status.

Once the target is resolved, the Routing Layer executes routing: Calls directed to a regular physical file are mapped directly to the underlying file system. Conversely, calls targeting a VDF are routed to the VDF engine, where read operations are sent to the Block Caching System and write operations to the Process Output Handler.

4.1 The VDF Engine

The fundamental design challenge of the VDF engine is hiding the latency of just-in-time computation behind standard file system semantics. To the user, a VDF appears as an immediately accessible file. In reality, accessing it triggers a cascading execution of lineage

¹The RepeatFS source code is available at <https://github.com/ToniWestbrook/repeatfs>.

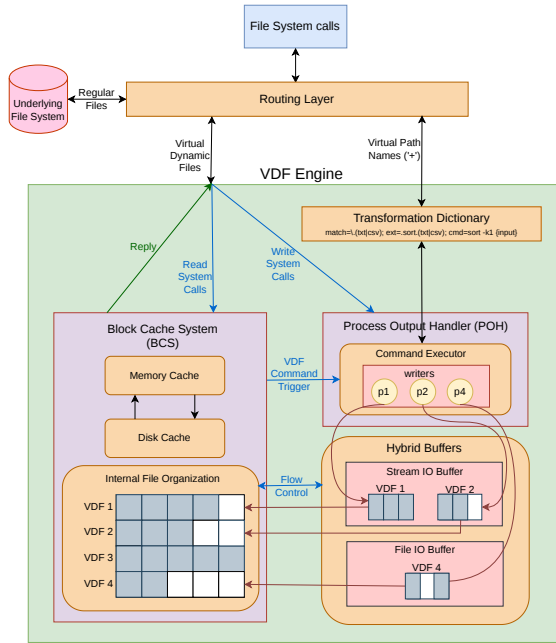


Figure 2: RepeatFS's point-and-click design

dependencies, as a single VDF often represents the output of a multi-step transformation. If the engine executed these steps sequentially, waiting for step N to complete and write to disk before starting step $N + 1$, the accumulated delay would break the interactive promise of the system.

Conversely, while executing the commands concurrently via a standard shell pipeline (e.g., $A|B$) solves the latency issue, it lacks the semantics required for a file. UNIX pipes are ephemeral (data is lost once consumed), FIFO (preventing random access or seeking), and incompatible with the fan-out pattern where an intermediate result feeds multiple downstream lineages. Therefore, the VDF engine implements a hybrid execution model: it launches all commands in the VDF's lineage workflow concurrently to stream results directly to the user, yet intercepts their I/O to enforce persistence, enable random access, and ensure consistency.

The VDF engine consists of four tightly coupled components: (1) the Transformation Dictionary, which serves as the registry of pre-defined transformation command templates; (2) the Block Caching System (BCS), which manages the dynamic metadata and data of virtual files; (3) the Process Output Handler (POH), which manages the execution lifecycle of the commands that generate VDFs; and (4) the Hybrid Buffer, which captures and paces the data generated by both stream-based and file-based executables.

In this section, we explain the internal architecture of the engine. In the next section, we present how these components integrate with the kernel to service standard file system calls.

4.1.1 The Transformation Dictionary. The Transformation Dictionary defines the potential topology of the virtual file system. It is

an in-memory registry, loaded at startup from a configuration file, whose rules use regular expressions to express flexible, many-to-one transformations (Table 1).

Each rule has three fields: a match regular expression defining the valid input types, an ext suffix appended to the resulting virtual file, and a cmd template with placeholders for input and output. For example, a stream-based sort and a file-based conversion are defined as:

```
match=\.txt|csv; ext=.sort.(txt|csv); cmd=sort -k1 {input}
match=\.csv; ext=.xlsx; cmd=ssconvert {input} {output}
```

The Dictionary is consulted whenever a virtual path is evaluated (ls, cd, getattr, open): the engine matches the parent file's extension against the registered rules and uses each matching ext to project a virtual entry (discovery) or authorize a child path (traversal). These regex rules thereby define the graph of valid transformations, revealing downstream options while excluding invalid ones (Figure 1).

4.1.2 The Block Caching System (BCS). The BCS serves as the data manager of the VDF engine. It is responsible for maintaining the data and metadata of all active VDFs, ensuring that dynamically generated virtual streams appear as standard, persistent files to the calling application. The materialized data of each VDF is stored in a two-tiered caching hierarchy: a memory cache and a disk cache. To prevent memory cache overflow during the processing of large datasets, the BCS implements an eviction policy. When the memory cache reaches a configurable threshold, the system flushes Least Recently Used (LRU) blocks to a secondary disk cache.

To track this data, the BCS maintains a Metadata Control Block (MCB) for every active VDF. Functioning analogously to a traditional kernel inode, the MCB maintains the execution state and cache pointers for the file, containing the following fields:

- **path:** The full transformation lineage string. When combined with a file block index, it forms a composite key that hashes into a global lookup dictionary to retrieve the VDF's cached content. This dictionary returns physical cache handles, which resolve to either a memory address (for active blocks) or a disk offset (for evicted blocks).
- **final:** A boolean state variable that is set to (True) if the virtual file is fully materialized.
- **flags:** An integer bitmask provided by the kernel during the open call, enforcing POSIX read/write access permissions.
- **current_size:** The dynamic, materialized file size. It defaults to a non-zero value prior to execution to prevent applications from aborting during size checks, and then dynamically tracks the growing stream size once materialization begins.
- **mtime:** The modification timestamp, initialized to match that of the physical source file, ensuring that directory listings present a logical time to the user.

BCS Validation and Population. To support lazy (just-in-time) evaluation, the engine separates cache validation from data materialization. When a VDF is created, its final flag is set to false, the mtime is initialized to the source file's mtime, and no cache blocks are allocated for it in the BCS.

Cache validation occurs when an application issues an open system call, creating a new file descriptor for the VDF. At this point, the VDF engine compares the `mtime` of the source file with the `mtime` recorded for the VDF. If they differ, indicating that the source has been modified since the virtual file was last generated, the cache is invalidated to guarantee causal consistency.

Rather than deleting the MCB, which would disrupt applications holding open file descriptors, the BCS resets the VDF state. Specifically, it sets `current_size` to 0, disconnects the associated memory and disk cache blocks, and resets the `final` flag to false. Importantly, the open operation only performs cache validation (and possible invalidation). The VDF is not populated at this stage and may never be populated if no subsequent read call occurs.

VDF data materialization is deferred until a read system call is issued. When a read request arrives, the BCS first checks whether the requested block is present in the cache:

- (1) If the full block is available, it is returned immediately, regardless of whether the VDF has been finalized (i.e., other blocks may still be pending).
- (2) If the block is missing or only partially available, the system examines the `final` state variable:
 - If `final` is true (the file has been fully generated), the system returns the available partial data or an end-of-file indication.
 - If `final` is false, the engine blocks the reader and signals the POH to start the worker process, if it is not already running. The reader remains suspended until the required block is generated and written to the cache.

Concurrency Control. The BCS utilizes a prioritized waiting queue when multiple processes are reading from the same actively generated VDF. Reads are served in block order - a reader requesting an earlier block is satisfied first while one requesting a later block waits - enforcing sequential generation and preventing thrashing.

4.1.3 The Process Output Handler (POH). Functioning as the execution engine of the system, the POH initiates, monitors, and terminates the commands that generate each VDF. When the BCS detects a cache miss on a VDF, it signals the POH to materialize the data.

Upon receiving an execution request, the POH first verifies the system state to ensure the requested VDF actually requires generation. Because the MCB's `final` boolean only tracks whether a file is finished, the POH must cross-reference two separate sources to determine the exact execution state. It checks its own internal registry to see if an active `writer_pid` already exists for the requested file (indicating it is actively materializing), and it queries the MCB's `final` flag to verify if the file is already complete (indicating it is finalized). If either of these conditions is true, the POH bypasses redundant execution. This distributed state check prevents duplicate execution if multiple concurrent cache misses target the same file. Once verified as a true cache miss, the POH retrieves the appropriate command template from the Transformation Dictionary. It dynamically expands this template, replacing logical placeholders with the physical paths of the input file and the target VDF, and then forks the concurrent worker process within its own shell session.

During process initialization, the POH records the parent shell Process ID (PID) of the forked worker in its internal active registry. This PID acts as an authorization token; only this specific process (or its child lineage) is permitted to write to the VDF, safeguarding the pipeline against unauthorized or malicious data injection from reused PIDs. Once the process is running, the POH redirects the tool's output to the Hybrid Buffer.

The POH utilizes the Hybrid Buffer to synchronize the writer's data generation rate with the reader's consumption rate. Rather than actively polling or signaling the worker process to pause, the POH relies on the capacity limits enforced by the buffer. Because standard memory buffers do not natively implement backpressure, the Hybrid Buffer wraps the underlying data structures with a custom, manually managed locking mechanism. When a buffer reaches its maximum capacity, these synchronization locks explicitly intercept the write request and block the writer process. Once the BCS retrieves and empties the buffered data via the unified interface, the locks are released, awakening the blocked process to resume execution. This shared blocking architecture ensures precise, uniform flow control across both stream-based and file-based tool outputs.

4.1.4 The Hybrid Buffer Architecture. A requirement of the VDF engine is compatibility with unmodified executables, regardless of their I/O behavior. Depending on how a transformation is defined, a tool's output is captured in one of two ways: from standard output (stream-based, as with the `sort` and `grep` transformations in Table 1) or from an explicit output file (file-based, as with `ssconvert`).

To accommodate both, the POH implements a Hybrid Buffer that adapts the tool's output to the block-based storage of the BCS. The POH writes the generated data to this buffer, while the BCS continuously reads from it. Importantly, the BCS always consumes data from the Hybrid Buffer sequentially, effectively treating all worker process output as a stream. Once the BCS consumes this data and materializes it into its block-aligned memory and disk caches, it can serve downstream read requests in any pattern. This architecture allows a downstream file-based executable to perform arbitrary, random-access `read()` system calls against the BCS cache, even while the upstream worker process is still sequentially generating the data.

Stream I/O Buffer (SIOB). For stream-based tools, the POH redirects `stdout` and `stderr` into the SIOB, a custom synchronized buffer that, unlike OS pipes or plain Python buffers, supports bidirectional blocking: a BCS read on an empty buffer blocks until the worker produces data, and a worker write to a full buffer blocks until the BCS drains it. This yields flow control aligned to the generation and consumption rates without relying on native file-system objects.

File I/O Buffer (FIOB). For file-based tools, the POH receives output through the FIOB. Since any process could issue a `write` or `truncate` against the VDF's path, the FIOB verifies that each call originates from the POH-spawned worker (or its child lineage) by matching PIDs and that the VDF is not yet finalized, rejecting all other writers with an access-denied error; the finalized check also guards against operating-system-reused PIDs. Verified data then flows into the same capacity-based buffering as the SIOB.

5 POSIX System Call Integration

RepeatFS bridges the point-and-click paradigm with the command line by mapping *Discovery* (`ls`), *Selection* (`cat`), and *Chaining* (`cd`) to standard POSIX file operations. The kernel translates each into FUSE callbacks; this section describes how the Routing Layer and the VDF engine handle them so the user sees an immediately accessible, pre-computed file system.

5.1 Directory Navigation and Listing (`cd`, `ls`)

The RepeatFS daemon is stateless with respect to the working directory: a `cd` issues no `chdir()`, and the kernel verifies the target via `getattr/access`, which RepeatFS resolves by parsing the path and returning the appropriate directory statistics and permissions. A `ls` issues `getattr`, `opendir`, and `readdir`; the `readdir` callback acts as a dynamic namespace generator. For a physical directory it scans the host directory and appends a virtual entry for every applicable transformation from the Transformation Dictionary; for a virtual directory it treats the parent VDF as the source and projects the next layer of applicable transformations. The user thus sees a directory listing whether interacting with source data or exploring nested workflows, even though the virtual entries do not exist on disk.

5.2 File Access and Streaming (`cat`)

The `cat` command represents the most complex interaction with the VDF engine. To illustrate how RepeatFS handles lineage, consider the chained request `cat data.csv+/data.grep.csv+/data.xlsx`, which triggers the following sequence:

- (1) Setup and Validation (`getattr`, `open`): The Routing Layer inherits the physical source's (`data.csv`) POSIX permissions to enforce access control and returns a spoofed virtual size (Section 4.1.2). During `open`, RepeatFS assigns a virtual file descriptor and evaluates the source's `mtime` for cache invalidation. To maintain lazy evaluation, no data is populated and no worker processes are spawned during this phase.
- (2) Materialization Trigger (`read`): The kernel requests data. RepeatFS queries the BCS; on a cache miss for a non-finalized file, the engine blocks the reader and signals the POH to spawn the background worker process for the terminal VDF (the `.xlsx` transformation).
- (3) Cascading Execution: The recursive generation of intermediate files is driven by the worker processes themselves. When the POH forks the worker for `data.xlsx`, that process attempts to read its upstream input by issuing standard OS `open()` and `read()` calls on `data.grep.csv`. These requests trap back into the FUSE Routing Layer, recursively repeating the validation and materialization sequence for the intermediate VDF. This, in turn, spawns the upstream sorting worker, which ultimately opens the physical `data.csv`. This elegantly constructs a concurrent, lazy-evaluated execution pipeline from the bottom up without ever blocking the FUSE daemon's main event loop.
- (4) Teardown (`release`): Once the `cat` command reaches the End-Of-File (EOF), the kernel issues a `close` system call, which FUSE translates to `release`. RepeatFS catches this call, removes the file descriptor from the active tracking

dictionary, and safely garbage collects the internal session references. This teardown cascades back up the chain as each process finishes reading its respective input, mimicking standard POSIX file lifecycle semantics. Crucially, while the ephemeral session is destroyed, the finalized MCB and materialized data remain persistently stored in the BCS cache to serve future requests instantly.

5.3 Restricted Operations

Because a VDF is the output of its lineage, in-place modification is inapplicable: a write or truncate from a process other than the authorized POH worker is rejected with `EACCES`. Metadata or namespace operations (`chmod`, `ln`, `mv`) are likewise rejected, since a VDF's permissions are inherited from its source and its location is fixed by its lineage.

6 Evaluation

A reproducibility system is usually evaluated on how faithfully it reproduces results and on the overhead of recording provenance. In RepeatFS's point-and-click model, reproducibility is a structural property rather than an empirical outcome: by construction, any VDF a user reaches is the recorded, consistent output of a valid workflow, produced without polluting the workspace and kept current with its source. Because these guarantees follow from the design, the evaluation does not ask whether they hold but what they cost. Materializing derived files just-in-time, streaming them through memory, and validating consistency each add overhead relative to plain disk I/O; our evaluation quantifies that cost, namely the latency and streaming efficiency of the VDF abstraction.

We benchmark RepeatFS against a traditional native disk I/O workflow, Snakemake [16] (representing automated execution), and ReproZip [6] (representing post-hoc provenance capture). Both Snakemake and ReproZip rely on disk-materialized execution models, introducing orchestration and tracking overheads, respectively. In contrast, RepeatFS embeds provenance capture into the file system, streaming intermediates in-memory while avoiding repeated disk materialization. We also compare against a native pipe streaming baseline as an idealized lower bound. These comparisons isolate the overhead of the VDF model, showing that RepeatFS approaches native streaming performance.

To support independent verification and adhere to FAIR [32] principles, all RepeatFS artifacts are available on Zenodo at DOI [30].

6.1 Read and Write Operations

Experiments were conducted on a Dell OptiPlex 7080 workstation (Intel Core i7-10700 CPU, 16 GB RAM) running Fedora Linux 40. Experimental data was placed on a 1 TB 7200 RPM HDD. To measure latency, we evaluated the time required to read, process, and output a 1 GB file, comparing RepeatFS's physical disk operations against its virtual VDF operations.

Figure 3 plots distinct execution scenarios for a 1 GB read/write. The Native baseline is a standard disk-bound operation. RepeatFS (cold) is the first access to a derived file: the POH executes the transformation, which must read the source from disk and compute, all through the FUSE layer. This regime cannot outperform native disk I/O, and does not - it is roughly 20% slower - consistent with the

expectation that a transformation reading from disk cannot beat a raw disk read. RepeatFS (cached) is a subsequent access served from the already-materialized in-memory BCS cache; avoiding both disk I/O and re-execution, it is about $3.5\times$ faster than a native read ($2.5\times$ for writes). The speedups therefore reflect avoided I/O and reuse of materialized results, rather than faster raw disk throughput. The systemic benefit of avoiding repeated disk materialization across a multi-stage pipeline is shown next.

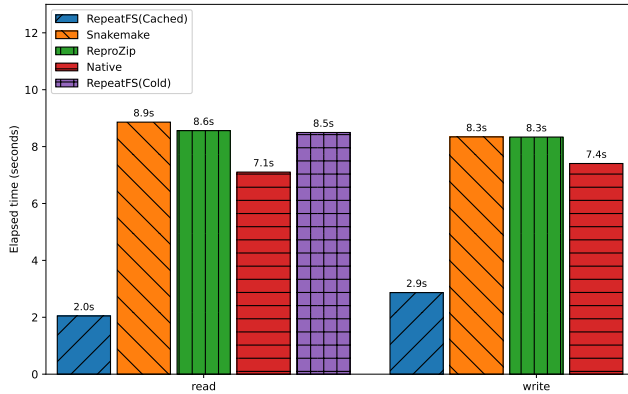


Figure 3: Read and write latency for a 1 GB file (block size 1 MB).

6.2 Synthetic Workflows

Using the same hardware, we assessed scalability by measuring latency as sequential processing stages scaled from one to ten. At each stage, a 1 GB file is read, processed, and written to serve as the next input. Figures 4a and 4b compare RepeatFS against the disk-reliant baselines and a theoretical optimal memory-piped baseline. To simulate real-world computational overhead, Figure 4b introduces a 10 ms/MB processing delay at each intermediate stage.

As expected, ReproZip and Snakemake exhibit linear latency growth and increasing variance because each stage must wait for previous mechanical disk flushes to complete, causing cumulative I/O contention. In contrast, RepeatFS maintains predictable, near-constant latency by leveraging concurrent in-memory data streaming, ultimately achieving a $4.5\times$ speedup over file-based workflows at ten stages.

6.3 Real-world Workflows

Real-world workflows were evaluated on a Dell PowerEdge R7625 HPC system (256 CPU cores, 3.0 TiB RAM, 45 TiB storage) running Rocky Linux 9.6. We evaluated four workflows chosen to represent a spectrum of CPU and I/O characteristics. These include *Bigrams*, a primarily I/O-bound text processing pipeline for linguistic modeling, and *NLP*, a CPU-intensive text parsing and lemmatization pipeline, both of which process Gutenberg project datasets [18]. Additionally, we tested *Encryption*, a CPU-bound cryptographic and compression pipeline, and *Paladin* [31], a CPU-intensive bioinformatics workflow dominated by metagenomic sequence alignment.

Figure 5 shows the mean workflow latencies. While ReproZip and Snakemake write all intermediate and output files to disk,

RepeatFS retains intermediate files in memory and the user only materializes the final output. Consequently, RepeatFS outperforms the disk-reliant systems across all tests, achieving speedups of $2.2\times$ for Bigrams, $1.7\times$ for NLP, $1.3\times$ for Encryption, and $1.1\times$ for Paladin. These results confirm that VDF streaming yields the largest performance gains for I/O-intensive workloads.

Summary:

The evaluation demonstrates that VDF workflows provide distinct advantages by maintaining virtual intermediates and performing access-based currency checks, thereby avoiding redundant disk I/O and producing notable speedups. This model also automates recomputation and eliminates the need for manual cleanup. However, the performance benefits are modest for workflows that are heavily CPU-bound, rely on very small intermediates, or require full stage completion prior to downstream processing (such as sorting operations). Additionally, the metadata management required for causal consistency introduces a small overhead that primarily affects extremely lightweight pipelines.

7 Discussion

We discuss the scope and limits of the point-and-click model: what it expresses and guarantees, the one structure it does not express and a path toward it, and how it interacts with the surrounding system in menu size, caching, and provenance.

7.1 Scope and Semantics

The model handles derivations expressible as a rooted tree over a single source, namely, linear chains and fan-out, with transformations parameterized by registering named variants (Table 1). Its guarantee is one of validity and consistency, not semantic correctness: because lineage is encoded in the path and kept current with its source, any accessible file is the reproducible output of a valid, recorded sequence of operations. The model does not judge whether that sequence is the scientifically intended one; selecting the meaningful workflow among the valid ones remains the researcher's responsibility.

Two properties follow from this design. Arbitrary parameters typed at access time are deliberately not admitted: the model permits only predefined atomic actions, as a GUI restricts users to its defined operations, so a new parameterization is registered in the configuration and a genuine one-off is performed outside the VDF tree. Immutability is likewise a defining property rather than a restriction: a VDF's content is by construction the output of its lineage, so it changes only when its source changes, and in-place mutation (write, chmod, ln, mv) is inapplicable; a writable artifact is obtained by materializing the VDF to a physical file. The one structure the model does not express is fan-in, which we address next.

7.2 Generalizing to Fan-In

The fan-in limitation lies in how lineage is navigated, not in the VDF abstraction itself. A path is rooted at a single source, so the directory tree cannot natively express many-to-one (fan-in) dependencies such as joins or multi-file builds. The VDF mechanism, however, is not intrinsically single-parent: just-in-time materialization, caching,

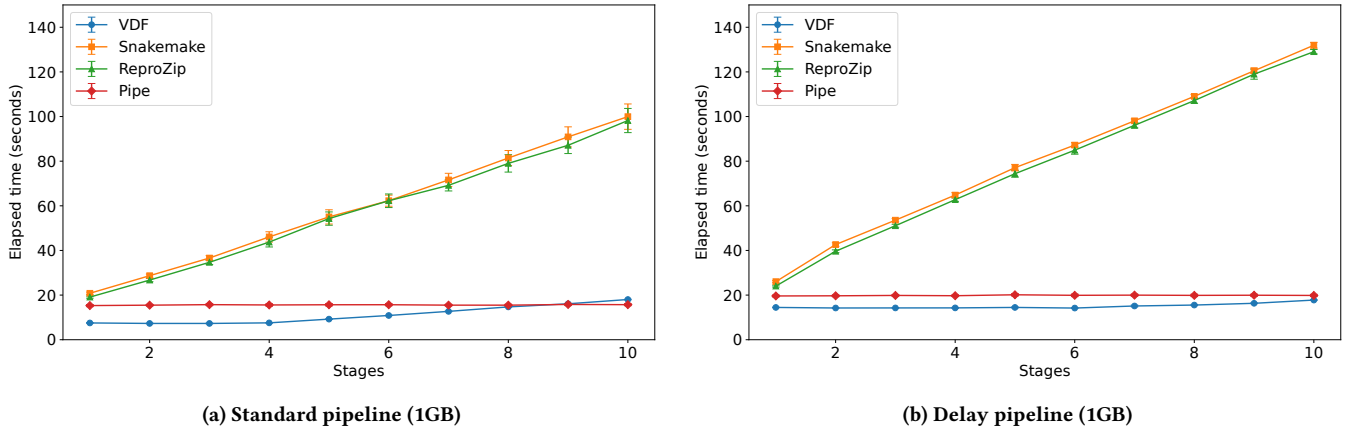


Figure 4: Pipeline latency versus number of workflow stages (primary and intermediate file sizes: 1 GB).

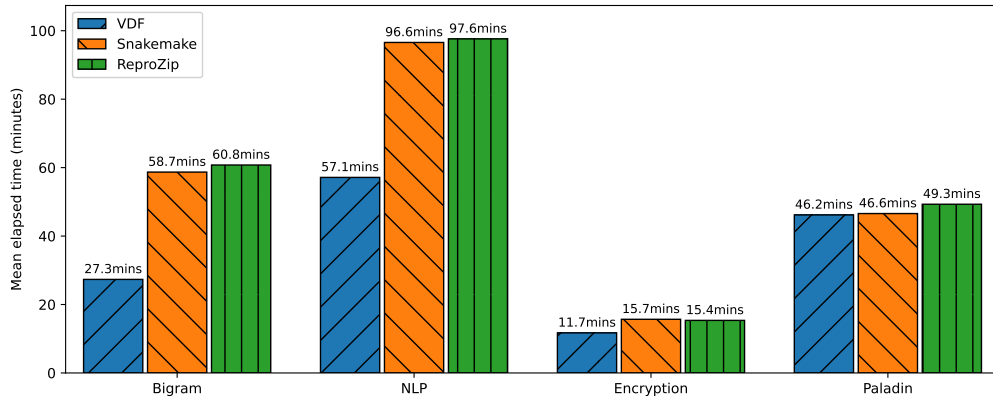


Figure 5: Performance comparison of RepeatFS against disk-reliant workflow managers (Snakemake, ReproZip) across real-world workloads. RepeatFS eliminates redundant disk I/O for intermediate files, yielding substantial latency reductions for I/O-intensive pipelines while maintaining competitive performance for CPU-bound tasks like Paladin.

and concurrent execution do not depend on a derived file having only one input, and a VDF can in principle be computed from several. What is bound to a single root is the way lineage is browsed and presented through the file system, not the way it is produced.

This separation suggests two levels of solution. The near-term path preserves the path-as-lineage invariant: a transformation takes a primary input (the path spine) plus named secondary inputs recorded in the invocation, so an arbitrary acyclic graph projects onto a primary spanning tree augmented with co-input edges. Single-input transformations remain fully discoverable by listing, while fan-in operands are named explicitly, so the metaphor degrades gracefully rather than completely. The longer-term direction is to make the VDF a multi-parent derivation, so that general graphs are supported by the abstraction directly; there the open question is not the materialization mechanism but the navigation and provenance presentation - how a graph, rather than a path, is browsed and recorded through the file system. Single-source derivation is the current implementation's scope.

7.3 Menu Manageability

The transformations projected for a file type are not all conceivable operations, but the curated subset of registered rules matching that type, re-evaluated at each node. Because the menu contains only what the researcher has registered, and only the variants applicable to the current type, its size is governed by the configuration rather than imposed by the system. For the focused, domain-specific configurations the system targets, this set is small and well suited to tree-style presentation; a larger menu simply reflects a larger registered set, under the user's control.

7.4 Caching and Filesystem Interactions

A VDF does not use the kernel's page cache. This is a choice, not a limitation: FUSE-served content can be page-cached even without a backing disk file, so a VDF could in principle be cached there. However, RepeatFS deliberately disables kernel caching for VDFs because the page cache cannot provide the control that a VDF requires.

A VDF is not merely data to be cached; it is the output of a transformation that must be coordinated with its readers. The BCS, together with the POH and Transformation Dictionary, ensures that a transformation runs only once even under concurrent access. It serves already-produced blocks while later blocks are still being generated and suspends readers requesting unavailable blocks until those blocks are produced, providing the pipe-like behavior central to VDF.

Thus, the page cache manages cached file data, whereas the BCS manages dynamically generated VDF blocks and the producer-consumer coordination they require, a role the page cache cannot fulfill.

The BCS uses a two-tier strategy: materialized blocks are held in a memory cache, and least-recently-used blocks spill to a BCS-managed disk region once a configurable threshold is exceeded. The replacement policy is a standard default, and any standard algorithm could be substituted.

Filesystem interactions are kept cheap by lazy materialization: listing, stat, glob expansion, and traversal validate paths but execute nothing, and only a read triggers computation, so expensive work is confined to data access rather than metadata traversal.

7.5 Provenance on Copy

Beyond just-in-time materialization, RepeatFS records operation-level provenance for all file operations on both physical files and VDFs, throughout a session [28, 29]. This record is complementary to the point-and-click model: the directory path makes a derivation navigable and re-executable on demand, whereas the provenance record preserves the derivation passively and independently of the path. Consequently, when a VDF is materialized to a physical file (Section 2.2.4), its lineage remains recoverable from this record.

8 Related Work

There are many types of computational reproducibility tools, each engineered to solve specific phases of the research lifecycle:

Computational Notebooks. Tools such as Jupyter Notebooks [15] and RMarkdown [34] are the standard for interactive, exploratory data science. They excel at blending narrative text, code, and in-line visualizations. However, they are susceptible to state drift and workspace pollution [23]. Because users can execute cells out of order, the visual layout of the notebook often does not reflect the actual execution lineage required to reproduce the current state in memory. RepeatFS targets a similar exploratory audience but enforces causal lineage at the file system level, ensuring that what the user sees on the screen is immutably tied to the exact transformations required to generate it.

Workflow Management Systems. Workflow engines such as Snakemake [16] and Nextflow [9], along with classic systems like Kepler [2] and Pegasus [8], are widely used for executing large-scale, declarative pipelines. They are highly generalized systems that support distributed scheduling, container orchestration, and parallel execution across diverse computing environments. However, they require users to explicitly specify and maintain dependencies using domain-specific languages, which can impose significant upfront structure during early, exploratory stages of analysis. RepeatFS is

not intended to replace the expansive distributed computing capabilities of these systems. Instead, it addresses the gap prior to workflow formalization, allowing scientists to chain tools using standard shell semantics and point-and-click interactions while still achieving pipeline-like automation and reproducibility. Furthermore, VDFs can complement existing workflow engines by enabling transparent, file-system-level streaming of intermediate data, reducing the need to materialize large intermediates sequentially on disk.

Post-hoc Container Packaging. Systems such as ReproZip [6], PTU [21], CARE [13] and CDE [10] provide post-hoc reproducibility by monitoring an already executed workflow. They trace system calls (typically via ptrace), capture data and library dependencies, and package them into a self-contained execution environment. Docker [4] and other container platforms provide lightweight, portable, reproducible containers. At the far end of this spectrum, Olive [24] uses virtual machine encapsulation to archive complete execution environments, enabling closed-source applications to be re-executed decades after their creation with a single mouse click. These tools are highly effective for sharing finalized experiments and ensuring they can be replayed on different hardware platforms. However, they do not assist users during the active research process. In contrast, RepeatFS trades the portability of standalone container bundles for continuous provenance tracking that is natively embedded in the file system.

Lineage-Aware File Systems. Provenance capture at the storage layer has been explored by systems such as the Provenance-Aware Storage System (PASS) [17]. PASS augments the operating system kernel to automatically record fine-grained file lineage as data is read and written, providing an accurate and comprehensive history of file derivations. These systems are designed for provenance tracking rather than reproducibility: they record how data was produced, but do not use that information to re-execute or maintain computations over time.

Existing solutions [7, 19, 22, 27, 35] excel in their respective domains, but none target the exploratory phase nor the issue of dynamically derived files. RepeatFS occupies a distinct niche: it is the first system, to our knowledge, designed specifically for transparent, point-and-click reproducibility.

9 Conclusion

In this paper, we introduced the point-and-click reproducibility model and implemented it in RepeatFS, a file system designed to bring the transparent reproducibility of menu-driven consumer applications to command-line data analysis. At the core of the system is the VDF abstraction, which encodes the lineage of derived files in the file system directory hierarchy, so that any file a user reaches is the recorded, consistent output of a valid workflow, preventing invalid, unrecorded operations and the state drift of stale derived files.

Currently, RepeatFS is implemented in user space as a FUSE daemon, requiring no kernel patches and thus remaining portable across systems. However, users must manually mount and unmount the file system during their sessions. Although our target users are

comfortable with command-line environments, this requirement introduces additional overhead into their day-to-day use.

We argue that the VDF abstraction is fundamentally a system-level property and would be most effective if integrated natively into the host operating system. Such a system could provide a default set of file transformation definitions via a startup configuration file. Users could modify this startup file or supply a session-specific configuration, allowing the VDF engine to automatically load these definitions at startup. This approach would tailor the file system to a researcher's domain without manual intervention.

The current version of the VDF engine supports only single-source derivations. However, complex scientific and computational workflows frequently exhibit many-to-one or many-to-many lineage relationships. For example, compiling a C executable depends on numerous source and header files, just as generating a PDF via $\LaTeX$ requires multiple .tex files and bibliographies. Our immediate future work will focus on extending the VDF abstraction to support multiple parent sources within a single derived lineage, enabling RepeatFS to capture and automate a broader class of complex data dependencies. Relatedly, the current implementation encodes a derived file's full lineage in its name, so file names and paths lengthen with pipeline depth; replacing this with the compact naming used in our examples is part of our future work.

Acknowledgments

We thank the shepherd and the anonymous reviewers for their careful reading and comments; incorporating their suggestions greatly improved this paper.

References

- [1] Enis Afgan, Dannon Baker, Bérénice Batut, Marius Van Den Beek, Dave Bouvier, Martin Čech, John Chilton, Dave Clements, Nate Coraor, Björn A Grüning, et al. 2018. The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update. *Nucleic Acids Research* 46, W1 (07 2018), W537–W544. doi:10.1093/nar/gky379
- [2] I. Altintas, C. Berkley, E. Jaeger, M. Jones, B. Ludascher, and S. Mock. 2004. Kepler: an extensible system for design and execution of scientific workflows. In *Proceedings, 16th International Conference on Scientific and Statistical Database Management, 2004*. IEEE Computer Society, USA, 423–424. doi:10.1109/SSDM.2004.1311241
- [3] Monya Baker. 2016. 1,500 scientists lift the lid on reproducibility. *Nature* 533 (2016), 452–454. doi:10.1038/533452a
- [4] Carl Boettiger. 2015. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review* 49, 1 (Jan. 2015), 71–79. doi:10.1145/2723872.2723882
- [5] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2001. Why and where: A characterization of data provenance. In *International Conference on Database Theory (ICDT '01)*. Springer, Springer-Verlag, Berlin, Heidelberg, 316–330.
- [6] Fernando Chirigati, Dennis Shasha, and Juliana Freire. 2013. ReproZip: using provenance to support computational reproducibility. In *Proceedings of the 5th USENIX Workshop on the Theory and Practice of Provenance (TaPP '13)*. USENIX Association, USA, Article 1, 4 pages.
- [7] Dong Dai, Yong Chen, Philip Carns, John Jenkins, and Robert Ross. 2017. Lightweight Provenance Service for High-Performance Computing. In *2017 26th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE Computer Society, Los Alamitos, CA, USA, 117–129. doi:10.1109/PACT.2017.14
- [8] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip J. Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and Kent Wenger. 2015. Pegasus, a workflow management system for science automation. *Future Generation Computer Systems* 46 (2015), 17–35. doi:10.1016/j.future.2014.10.008
- [9] Paolo Di Tommaso, Maria Chatzou, Evan W Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. 2017. Nextflow enables reproducible computational workflows. *Nature Biotechnology* 35, 4 (2017), 316–319. doi:10.1038/nbt.3820
- [10] Philip J. Guo and Dawson Engler. 2011. CDE: Using System Call Interposition to Automatically Create Portable Software Packages. In *2011 USENIX Annual Technical Conference (USENIX ATC 11)*. USENIX Association, Portland, OR, 21. <https://www.usenix.org/conference/usenixatc11/cde-using-system-call-interposition-automatically-create-portable-software>
- [11] IEEE. 2017. *IEEE Std 1003.1-2017: Standard for Information Technology—Portable Operating System Interface (POSIX)*. IEEE, New York, NY, USA.
- [12] Robert Ikeda and Jennifer Widom. 2009. *Data Lineage: A Survey*. Technical Report. Stanford University. <http://ilpubs.stanford.edu:8090/918/>
- [13] Yves Janin, Cédric Vincent, and Rémi Duraffort. 2014. CARE, the comprehensive archiver for reproducible execution. In *Proceedings of the 1st ACM SIGPLAN Workshop on Reproducible Research Methodologies and New Publication Models in Computer Engineering (Edinburgh, United Kingdom) (TRUST '14)*. Association for Computing Machinery, New York, NY, USA, Article 1, 7 pages. doi:10.1145/2618137.2618138
- [14] Sean Kandel, Andreas Paepcke, Joseph M. Hellerstein, and Jeffrey Heer. 2012. Enterprise Data Analysis and Visualization: An Interview Study. *IEEE Transactions on Visualization and Computer Graphics* 18, 12 (2012), 2917–2926. doi:10.1109/TVCG.2012.219
- [15] Thomas Kluyver, Benjamin Ragan-kelley, Fernando Pérez, Brian E. Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. 2016. Jupyter Notebooks—a publishing format for reproducible computational workflows. 87–90 pages. doi:10.3233/978-1-61499-649-1-87
- [16] Johannes Köster and Sven Rahmann. 2012. Snakemake: a scalable bioinformatics workflow engine. *Bioinformatics* 28, 19 (Aug. 2012), 2520–2522. doi:10.1093/bioinformatics/bts480
- [17] Kiran-Kumar Muniswamy-Reddy, David A Holland, Uri Braun, and Margo Seltzer. 2006. Provenance-aware storage systems. In *Proceedings of the Annual Conference on USENIX '06 Annual Technical Conference*. USENIX Association, USA, 43–56.
- [18] NimiQ. 2014. gutenburg-bulk-downloader. <https://github.com/nimiQ/gutenberg-bulk-downloader>.
- [19] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eysers, Margo Seltzer, and Jean Bacon. 2017. Practical whole-system provenance capture. In *Proceedings of the 2017 Symposium on Cloud Computing (Santa Clara, California) (SoCC '17)*. Association for Computing Machinery, New York, NY, USA, 405–418. doi:10.1145/3127479.3129249
- [20] Roger D. Peng. 2011. Reproducible Research in Computational Science. *Science* 334, 6060 (2011), 1226–1227. doi:10.1126/science.1213847
- [21] Quan Pham, Tanu Malik, and Ian Foster. 2013. Using Provenance for Repeatability. In *5th USENIX Workshop on the Theory and Practice of Provenance (TaPP '13)*. USENIX Association, Lombard, IL, 4 pages. <https://www.usenix.org/conference/tapp13/technical-sessions/presentation/phan>
- [22] Devin J. Pohly, Stephen McLaughlin, Patrick McDaniel, and Kevin Butler. 2012. Hi-Fi: collecting high-fidelity whole-system provenance. In *Proceedings of the 28th Annual Computer Security Applications Conference (Orlando, Florida, USA) (ACSAC '12)*. Association for Computing Machinery, New York, NY, USA, 259–268. doi:10.1145/2420950.2420989
- [23] Adam Rule, Aurélien Tabard, and James D. Hollan. 2018. Exploration and Explanation in Computational Notebooks. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (Montreal QC, Canada) (CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3173574.3173606
- [24] Mahadev Satyanarayanan, Jan Harkes, and James Blakley. 2023. Towards Reproducible Execution of Closed-Source Applications from Internet Archives. In *Proceedings of the 2023 ACM Conference on Reproducibility and Replicability (ACM REP '23)*. Association for Computing Machinery, New York, NY, USA, 15–26. doi:10.1145/3589806.3600035
- [25] Shneiderman. 1983. Direct Manipulation: A Step Beyond Programming Languages. *Computer* 16, 8 (1983), 57–69. doi:10.1109/MC.1983.1654471
- [26] M Szeredi. 2005. Filesystem in Userspace. <http://fuse.sourceforge.net>.
- [27] Dawood Tariq, Maisem Ali, and Ashish Gehani. 2012. Towards automated collection of application-level data provenance. In *Proceedings of the 4th USENIX Conference on Theory and Practice of Provenance (Boston, MA) (TaPP'12)*. USENIX Association, USA, 16. <https://www.usenix.org/conference/tapp12/workshop-program/presentation/Tariq>
- [28] Anthony Westbrook. 2021. *RepeatFS: A File System Providing Reproducibility Through Provenance and Automation*. Doctoral dissertation. University of New Hampshire. <https://scholars.unh.edu/dissertation/2640>
- [29] Anthony Westbrook, Bocheng Cui, Lakshmi Ongolu, and Elizabeth Varki. 2026. Active Reproducibility for File-Based Exploratory Science. In *Proceedings of the 2026 ACM Conference on Reproducibility and Replicability (ACM REP '26)*. Association for Computing Machinery, New York, NY, USA, 12 pages. 10.1145/3820002.3828580.
- [30] Anthony Westbrook, Lakshmi Ongolu, Bocheng Cui, and Elizabeth Varki. 2026. *RepeatFS Point-and-Click Experimental Setup*. doi:10.5281/zenodo.21041945

- [31] Anthony Westbrook, Jordan Ramsdell, Taruna Schuelke, Louisa Normington, R Daniel Bergeron, W Kelley Thomas, and Matthew D MacManes. 2017. PAL-ADIN: protein alignment for functional profiling whole metagenome shotgun data. *Bioinformatics* 33, 10 (05 2017), 1473–1478. doi:10.1093/bioinformatics/btx021
- [32] Mark D Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E Bourne, et al. 2016. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data* 3, 1 (2016), 160018. doi:10.1038/sdata.2016.18
- [33] Kanit Wongsuphasawat, Yang Liu, and Jeffrey Heer. 2019. Goals, Process, and Challenges of Exploratory Data Analysis: An Interview Study. arXiv:1911.00568 [cs.HC] <https://arxiv.org/abs/1911.00568>
- [34] Yihui Xie, JJ Allaire, and Garrett Golemund. 2018. *R Markdown: The Definitive Guide* (1st ed.). Chapman and Hall/CRC, Boca Raton, Florida. doi:10.1201/9781138359444
- [35] Michael Zipperle, Florian Gottwalt, Elizabeth Chang, and Tharam Dillon. 2022. Provenance-based Intrusion Detection Systems: A Survey. *ACM Comput. Surv.* 55, 7, Article 135 (Dec. 2022), 36 pages. doi:10.1145/3539605